

AI Response Evaluator - Technical Documentation

1. Project Overview

The **AI Response Quality Evaluator** is a comprehensive multi-agent platform designed to systematically score, judge, and audit responses generated by Large Language Models (LLMs). It automates the QA (Quality Assurance) process for LLM applications by using a panel of specialized judge agents.

2. Objectives

- Eliminate manual, subjective review of AI outputs.
- Provide a consistent, deterministic framework for scoring Relevance, Accuracy, and Completeness.
- Detect hallucinations and unsupported claims against ground-truth contexts.
- Support batch evaluations and comprehensive report generation.

3. System Architecture

The platform is built on a modern Python tech stack:

- **Backend:** FastAPI, providing async endpoints for evaluation.
- **Frontend:** Streamlit, serving an interactive dashboard.
- **Database:** SQLite (via SQLAlchemy) for local persistence of evaluation histories.
- **Agents:** Modular heuristic/LLM-backed Python classes representing different judging dimensions.
- **RAG:** FAISS (Vector DB) + HuggingFace SentenceTransformers for semantic retrieval.

4. Folder Structure

```
ai-response-evaluator/
├── backend/
│   ├── app/
│   │   ├── agents/           # Evaluation modules (Relevance, Accuracy,
│   │   ├── api/              # FastAPI routers
│   │   ├── database/         # SQLite setup and models
│   │   ├── rag/              # Document indexing and retrieval logic
│   │   ├── schemas/          # Pydantic validation models
│   │   ├── services/         # Orchestration layer
│   │   └── main.py           # FastAPI entry point
│   └── frontend/
│       ├── app.py            # Streamlit application UI
│       └── report_generator.py # PDF compilation logic
├── tests/                    # Pytest suite
├── docs/                     # Technical documentation
└── requirements.txt
```

5. Agent Modules

- **RelevanceJudge:** Calculates term-frequency overlap between question and response, enforcing length penalties.
- **AccuracyJudge:** Uses sentence-transformers to calculate cosine similarity against a reference answer or retrieved RAG context.
- **CompletenessJudge:** Cross-references the question's entities against the response to identify missing elements (omissions).
- **VerdictAgent:** Aggregates individual scores into a weighted final grade (A/B/C/F) and Pass/Needs Improvement/Fail verdict.

6. Installation & Execution

```
# 1. Install Dependencies
pip install -r requirements.txt

# 2. Run the FastAPI Backend
py -m backend.app.main

# 3. Run the Streamlit Frontend (in a new terminal)
py -m streamlit run frontend/app.py
```

7. Limitations & Future Work

Limitations: Current heuristic-based NLP agents (TF-IDF, simple Cosine Similarity) are less nuanced than LLM-as-a-judge patterns. They guarantee deterministic consistency but lack deep semantic reasoning. **Future Work:**

1. Integrate Gemini/OpenAI API as the judging engine.
2. Expand dimension agents (e.g., Tone Judge, Safety/Toxicity Judge).
3. Cloud database deployment (PostgreSQL).